

Load Testing (CRM CASA 360 / Customer 360, CBS Cash Deposit)

Browser-based load test harness built on Playwright. Each virtual user (VU) is a real Chromium session that authenticates **exactly once** and then loops the business flow over its own data records until the test duration or iteration cap is reached. Every business step is timed as a named transaction, and all samples are aggregated into an industry standard report (percentiles, throughput, error rate, Apdex, SLO verdict, time series).

1. Prepare test data

Test data lives in CSV files, one row per data set. Rows are **grouped by username/password**: a VU is assigned one user group, logs in once as that user and then cycles round-robin only through that user's records. This guarantees the session and the data always belong to the same user.

- `fixtures/casa360.csv` — `username,password,casaNumber`
- `fixtures/customer360.csv` — `username,password,customerId`
- `fixtures/cbs_cashDeposit.csv` —
`username,password,accountNo,amount,narrative,noteDenomination,authorizer`

Lines starting with `#` are comments. Values may be quoted if they contain commas. Every row must carry `username` and `password`.

Use distinct application users where possible. VUs are assigned users round-robin, so with 25 VUs and 2 users each user carries ~13 concurrent sessions, which can serialise server-side state and distort results. Aim for one user per VU.

Override the paths with `CASA360_DATA` / `CUSTOMER360_DATA` / `CASHDEPOSIT_DATA`.

Cash Deposit data rules

`amount` must be exactly divisible by `noteDenomination`; the note count handed to the denomination screen is derived as `amount / noteDenomination` (see `utils/cbs.helpers.js`). A mismatch is rejected before the run starts rather than producing a flood of misleading CBS validation errors.

This flow writes data. Every iteration posts and authorises a real cash deposit. A 10 VU / 10 minute run can create thousands of postings and will move the teller till and account balances. Only run it against UAT/DR with explicit sign-off, and agree the data clean-up beforehand.

2. Run

```
# validation run first: 2 VUs per flow, 2 iterations each
npm run load:smoke

# full run with the configured profile
LOAD_TEST=1 VUS=25 RAMP_UP_SEC=300 DURATION_SEC=1800 npm run load
```

```
# single flow
npm run load:casa360
npm run load:customer360

# CBS cash deposit – posts real transactions, never included by default
npm run load:cashdeposit:smoke      # 1 VU, 1 deposit, validates the flow
end to end
LOAD_TEST=1 FLOWS=cashDeposit VUS=5 RAMP_UP_SEC=60 DURATION_SEC=600 \
  npx playwright test tests/load
```

`npm run load` targets `tests/load` only. Regular functional runs (`npm test`) exclude the load specs automatically.

3. Configuration (all environment variables)

Variable	Default	Meaning
<code>BASE_URL</code>	<code>https://dr-crm.esafbank.org</code>	CRM entry point under test
<code>TEST_ENV</code>	<code>dr</code>	Free-text environment label stamped on the report
<code>CBS_BASE_URL</code>	<code>https://dr-cbs- uat.esafbank.org:8443/DashboardNG.html</code>	CBS entry point under test
<code>FLOWS</code>	<code>casa360,customer360</code>	Comma separated flows: <code>casa360</code> , <code>customer360</code> , <code>cashDeposit</code>
<code>VUS</code>	<code>10</code>	Virtual users per flow
<code>VUS_CASA360 / VUS_CUSTOMER360 / VUS_CASHDEPOSIT</code>	<code>VUS</code>	Per-flow VU override
<code>CASHDEPOSIT_TASK_CODE / CASHDEPOSIT_TASK_LABEL</code>	<code>1401 / Cash Deposit (1401)</code>	CBS task to open
<code>RAMP_UP_SEC</code>	<code>60</code>	VUs are started linearly across this window
<code>DURATION_SEC</code>	<code>600</code>	Steady-state duration per VU

Variable	Default	Meaning
ITERATIONS	0 (duration driven)	Hard cap on iterations per VU
THINK_TIME_MIN_MS / THINK_TIME_MAX_MS	1000 / 3000	Random pause between iterations
HEADLESS	true	Set false to watch the browsers
WORKERS	= total VUs	Playwright worker processes (one browser each)
ABORT_ON_ERROR	false	Fail the whole VU on the first transaction error
APDEX_T_MS	3000	Apdex satisfied threshold (tolerating = 4T)
SLO_P95_MS	5000	SLO: p95 response time
SLO_P99_MS	8000	SLO: p99 response time
SLO_ERROR_RATE_PCT	2	SLO: maximum error rate
SLO_MIN_TPS	0 (disabled)	SLO: minimum throughput
RESULTS_DIR	./load-results	Root output folder
LOAD_RUN_ID	timestamp	Run identifier / output sub-folder
REPORT_BUCKET_SEC	10	Time-series bucket width

Capacity note

One VU is one Chromium instance. Budget roughly 250–400 MB RAM and ~0.3 vCPU per VU. Beyond ~20–25 VUs on a laptop, the client becomes the bottleneck and the numbers measure your machine, not the CRM. Distribute across agents for higher concurrency.

4. Transactions measured

CASA 360 — 00 Login, 01 Search account, 02 Open account details, 03 Financial Details tab

Customer 360 — 00 Login, 01 Search customer, 02 Open customer details, 03 Communication tab, 04 KYC Info tab

CBS Cash Deposit — 00 Login, 01 Open task 1401, 02 Enter transaction, 03 Denomination details, 04 Authorise

Session model

00 Login is executed once per VU, before the iteration loop, and is recorded as its own transaction so it never inflates the per-iteration numbers. The session is then reused for every iteration.

After a failed iteration the harness checks whether the CRM login form has reappeared:

- **Session alive** — nothing happens, the next iteration continues on the same session.
- **Session lost** — a 00 Re-login (session recovery) transaction is recorded and the VU re-authenticates. If that also fails, the VU ends.

Re-login counts are printed per VU at the end of the run and visible as a separate row in `transactions.csv`. A non-zero count is a finding in itself: it usually means the server dropped sessions under load.

CBS additionally signs out once per VU after the last iteration, so teller sessions are released cleanly instead of being killed with the browser.

Each transaction ends only once the UI has stopped mutating (`waitForUiIdle`), so the timing covers server response plus rendering. Additional synthetic samples are written per iteration: **ITERATION** (end-to-end), **PAGE LOAD** (navigation timing) and **HTTP** (network summary).

5. Outputs

Everything lands in `load-results/<runId>/`:

Artefact	Contents
<code>index.html</code>	Self-contained dashboard: KPIs, SLO verdict, trend charts, all tables
<code>summary.json</code>	Full machine-readable report (CI gates, trend databases)
<code>transactions.csv</code>	Per transaction: count, pass/fail, min/avg/median/p90/p95/p99/max, std dev, TPS, Apdex
<code>flows.csv</code>	Same statistics rolled up per flow

Artefact	Contents
<code>timeseries.csv</code>	Per interval: active VUs, throughput, errors, avg and p95 response time
<code>samples.csv</code>	Raw sample log, one row per transaction execution
<code>errors.csv</code>	Grouped failures with counts and first/last seen
<code>http-metrics.csv</code>	Requests, 4xx, 5xx, failed requests, MB received, slowest request
<code>browser-metrics.csv</code>	TTFB, DOM interactive, DCL, load, FCP (avg and p95)
<code>screenshots/</code>	Screenshot per failed iteration
<code>raw/*.jsonl</code>	Per-VU sample streams (source of truth for aggregation)
<code>playwright-results.json</code> , <code>playwright-html/</code>	Native Playwright execution report

The report is generated automatically in the global teardown. To rebuild it without re-running the test:

```
npm run load:report # latest run
LOAD_RUN_ID=20260730-171500 npm run load:report
```

6. Reading the results

- **Error rate** — first thing to check. High error rates invalidate response times.
- **p95 / p99** — the headline SLO numbers; averages hide the tail.
- **Throughput vs active VUs** — if throughput flattens while VUs keep rising, the system has saturated; the knee of that curve is the capacity limit.
- **Std dev** — large values indicate unstable response times (queueing, GC, locks).
- **Apdex** — single user-satisfaction score, ≥ 0.94 is generally "good".
- **HTTP 5xx / failed requests** — server-side saturation signals.
- **SLO verdict** — **PASS/FAIL** computed from the **SLO_*** thresholds.

7. Report a run alongside server-side metrics

Client-side numbers alone are not a diagnosis. Capture CRM application server, DB and infrastructure metrics (CPU, memory, GC, thread pools, slow queries) over the same window and attach them to the run folder before sharing results.